Keyword Modifier3

- Modifier3 digunakan apabila suatu Class atau Interface memanfaatkan (**extends**) class pengguna atau mengimplementasikan (**implements**) Interface lain.
- **Merajut class-class yg membentuk struktur taxonomy dalam sebuah project.**

- Cara memanfaatkan Class atau Interface lain:
- 1. Dengan **instansiasi objek** dari class lain
- 2. Dengan **extends** class pengguna
- 3. Dengan **implements** Interface lain
- 4. class pengguna menjadi **tipe data attribute** pada Class yang memanfaatkan.
- 5. class pengguna menjadi **tipe data parameter** pada Class yang memanfaatkan.

6. class pengguna menjadi **tipe data return value** pada Class yang memanfaatkan.

extends

- Bila terjadi pewarisan, Class yang mewariskan method() dan attributenya disebut super Class, sedangkan yang diwariskan disebut subclass.

- Ada method() dan/atau attribute yg diwariskan dari super class ke subclass. Subclass menyatakan salah-satu “jenis” dari superclass, bukan “bagian” dari superclass.

- Class yang memiliki modifier3 **extends** artinya subclass dari suatu super class.

```
import namaPackage.BidangDatar2D;
public class PersegiPanjang extends BidangDatar2D {
}
```

implements

- Class yang memiliki modifier3 implements artinya class tersebut mengimplementasikan satu atau lebih interface. Bila yang diimplementasikan lebih dari satu interfce, maka digunakan tanda koma di antara nama-nama interface tersebut.

```
[modifier1] class Robot implements Manusia, Mesin {
    berpikir() {……?}
    menari() {……?}
    bekerjatanpakenallelah() {…?}
}
interface Manusia {
    String berpikir(int x);
    menari();
}
interface Mesin {
    String bekerjatanpakenallelah(int x);
}
```

Catatan

- Bila Class yang memanfaatkan dan menggunakan class pengguna, **tidak berada dalam satu package/directory/folder** dengan Class atau Interface yang dimanfaatkan,
- maka Class yang memanfaatkan/menggunakan harus meng**import** Class atau Interface yang dimanfaatkan/digunakan.

```
interface Geometri {
  public float menghitungKelilingLingkaran();
  public float menghitungKelilingLingkaran(float r);
  public float menghitungLuasLingkaran();
  public float menghitungLuasLingkaran(float r);
  public float menghitungVolume();
  public float menghitungVolume(float r);
}
```

```
public class Lingkaran implements Geometri {
    public static final double PI = 3.14159265359;
    public float kelilingLingkaran, luasLingkaran;
    public float r = 5.0;
    public float menghitungKelilingLingkaran(){
        kelilingLingkaran = 2*PI*r;
        return kelilingLingkaran;
    }
    public float menghitungKelilingLingkaran(float r) {
        //this.r = r;
        kelilingLingkaran = 2*PI*r;
        return kelililingLingkaran;
    }
    public float menghitungLuasLingkaran() {
        luasLingkaran = PI*r*r;
        return luasLingkaran;
    }
    public float menghitungLuasLingkaran(float r) {
                        //this.r = r;
                        return = PI*r*r;
    }
```

```
public class Tabung extends Lingkaran (
    //public float r = 0;
    public double tinggi = 0;
    public double luastabung = 0;
    public double volTabung = 0;
   // ada 3 cara untuk menghitung luas dan volume tabung
    public double hitungLuasTabung() {
        Lingkaran jj = new Lingkaran();
        luasTabung = 2*Lingkaran.PI*jj.r*tinggi;                    //cara1a
        luasTabung = 2*Lingkaran.PI*super.r*tinggi;                 //cara1b
        luasTabung = super.menghitungKelilingLingkaran()*tinggi;    //cara2a
        luasTabung = super.menghitungKelilingLingkaran(9.0)*tinggi; //cara2b
        luasTabung = super.kelilingLingkaran*tinggi;                //cara3
    }

    public double hitungVolumeTabung() {
        Lingkaran jj = new Lingkaran();
        volTabung = Lingkaran.PI*jj.r*jj.r*tinggi;                  //cara1a
        volTabung = Lingkaran.PI*super.r*super.r*tinggi;            //cara1b
        volTabung = super.menghitungLuasLingkaran()*tinggi;         //cara2a
        volTabung = super.menghitungLuasLingkaran(15)*tinggi;       //cara2b
        volTabung = super.luasLingkaran*tinggi;                     //cara3
    }
}
```

Sepuluh cara memanggil/mengakses suatu attribute/method() milik class pengguna:

1. namaObjek.namaMethod([argumen]);
2. namaObjek.namaAttribute;

3. namaClass.namaMethod([argumen]);
4. namaClass.namaAttribute;

5. super.namaMethod([argumen]);
6. super.namaAttribute;

7. this.namaMethod([argumen]);
8. this.namaAtribute;

9. namaMethod([argumen]);
10. namaAttribute;

Tidak semua cara akses dapat dilakukan

- Komponen Class yg juga disebut “member” yaitu **Class itu sendiri**, **attribute**, **constructor()**, **method()**, dan **inner Class** dapat diakses/dipanggil method() lain, dengan salah-satu dari format sebagai berikut:

Aturan Mengakses Member/Komponen class pengguna

method() yang diakses	method() yang mengakses misal main()
Static: Menggunakan *class*	*Static* atau *Non static*
Non static: Menggunakan *objek*	*Static*
Non static: Menggunakan *objek* Menggunakan *super* Menggunakan *this*/langsung	*Non static:* Di luar class yg diakses dan di luar sub class Di sub class Di dalam class yg sama

Parameter Var-Arg

- Argument adalah literal/nilai yang dimasukkan ke mathod() atau constructor() ditampung oleh parameter.
- Parameter adalah tempat untuk menampung argument tersebut.
- Var-Arg biasa digunakan sebagai parameter suatu method() ataupun constructor() namun boleh dimasuki argument dengan kuantitas yang bervariasi.
- Batasan-batasan pengunaan Var-Arg:
- 1. Hanya boleh digunakan 1x dalam 1 method().
- 2. Jika dicampur dengan parameter lain, maka hanya
- boleh digunakan pada urutan terakhir dalam
- daftar parameter.
- 3. Tidak boleh digunakan sebagai **tipe** suatu attribute.

Contoh penggunaan argument dan parameter

- **public void setData(String nama, String… alamat) {**
- …
- **}**

- **Pemanggilan** method() tersebut dengan cara:
- ***setData("Anin","Yogyakarta","Denpasar","Bantul");***

- Contoh di atas adalah suatu method() yang memiliki dua buah **parameter** yaitu **nama** dan **alamat** yang masing-masing bertipe String.
- Parameter nama dan alamat untuk menerima **argument** "Anin" dan "Yogyakarta".

Aturan-aturan Penggunaan Var-Arg

1. Tipe Var-Arg sama dengan variabel biasa, boleh berupa tipe apa saja, baik ***tipe data primitif***, ***class***, maupun ***array***.
2. Format sintaks harus benar, parameter yang merupakan var-arg wajib memakai tanda "**...**" tepat di belakang ***tipe*** parameter var-arg nya.
3. Penggunaan var-arg boleh ***dicampur*** dengan parameter yang lain, ditempatkan paling belakang.

Contoh:

```
setData(String... x);
setArray(String[]... y);
namaMethod(int jumlah, String... z);
setTranskripNilai(int nim, String nama, int... nilai);
setTranskripNilai(int... nilai, int nim, String nama); //salah
setTranskripNilai(String... nim, int nilai...); //salah
```

contoh seter() dan geter()

```
public class Cat {
    private String name;
    private int weight;
    private int height;
    private char jk;
    //int x;
    private int age;
    private byte belang;
    private String ras;
    public  void setWeight(int weight){
        //int x=5;
        //this.x=x;
        this.weight = weight;
    }
    public static void setAge(int age){
        this.age = age;
    }
    public int getWeight() {
        return weight;
    }
}
```

```
public class TesCat{
    public static void main(String[] args) {
        Cat jahil = new Cat();
        jahil.setWeight(2000);//dalam gram
        System.out.println("Cat's weight is = "
        +jahil.getWeight());
    }
}
```

setWeight() adalah method yg **dipanggil** method main().

main() adalah method yg **memanggil** method setWeigth()

Overloading terhadap method()

- ***Overloading*** adalah membuat beberapa method() dengan nama yg **sama** di dalam class yang **sama**, namun harus dengan memodifikasi deklarasi parameter sebagai pembeda.

- Method-method yg ***overloading*** wajib berada di dalam class yg sama atau salah-satunya di dalam superclass dan yg lain di dalam subclass.

- ***Overloading*** di OOP berbeda dengan overloading terhadap operator di C++.

- Interpreter/compiler mampu memilih method() mana yang dieksekusi dengan mengenali tipe parameter yg dilewatkan ke mthod() dan mengenali return valuenya.

Contoh Overloading

- class ContohOverloading {
- pangkat(int a, int b) {
- {
- pangkat(int a, float b) {
- {
- pangkat(float a, int b) {
- {
- pangkat(float a, float b) {
- {
- }

Keyword return

- Keyword ini digunakan untuk keluar dari suatu blok **method()**.
- Baris-baris kode program **setelah** keyword ini yang masih di dalam blok suatu mehod() tersebut **akan diabaikan**, kemudian eksekusi dilanjutkan ke baris kode setelah penutup blok method() tersebut.
- Contoh cuplikan kode program:

```
int abs(int x) {
    if (x >= 0) {

        return x;
    else {
        return (-x);
        x *= 10;
    }
}
```

- .....
- Catatan: coba keyword *break* dan *continue*.

constructor()

Pada prinsipnya:

Constructor() adalah method() yang tergolong setter(() yang dieksekusi saat instansiasi objek dan inisialisasi yang diperlukan. Secara implisit adalah instant dari class nya.

Nama constructor() harus **sama** dgn nama class-nya, dan boleh diberi modifier akses (public, protected, default, private) sebagaimana method() biasa.

```
[modifier1] NamaConstructor([type namaParameter]) {
    constructorBody;
}
```

Deklarasi Objek

- Objek adalah instansiasi (perwujudan) suatu class. Suatu objek terlebih dahulu harus **dideklarasikan**, setelah itu dibuat/**diujudkan**, setelah itu objek boleh **digunakan**, dan terakhir **dihancurkan**.

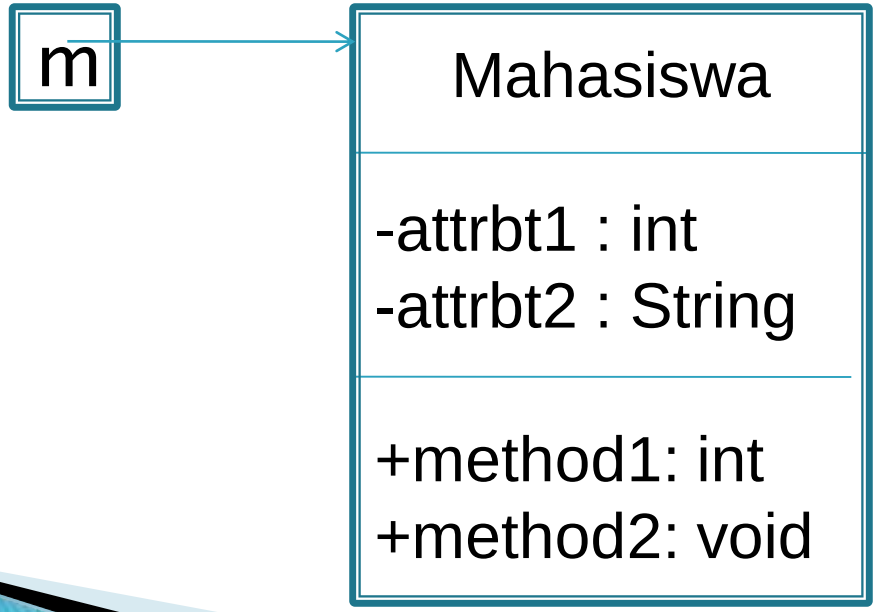
- Contoh deklarasi objek:
- **Mahasiswa m;**

Mahasiswa

- Ditentukan objek m dengan tipe Mahasiswa, saat deklarasi, m masih berupa pointer namun belum menunjuk ke suatu lokasi memori.

Instansiasi Objek

m = new Mahasiswa();



Empat cara coding untuk Deklarasi dan Instansiasi Objek

- **tipedata variabel;**
- Cara1:
- NamaClass namaObjek; //deklarasi
- **namaObjek = new NamaConstructor([parameter]); //instansiasi**
- Statement deklarasi dan instansiasi suatu objek boleh dipisah, bahkan boleh ditempatkan di class yang berbeda.

- Cara2:
- **NamaClass namaObjek = new NamaConstructor([parameter]);**

- Cara 3:
- **new NamaConstructor([parameter]);**
- Cara ini objek diinstansiasi tanpa nama, tujuannya hanya untuk memanggil constructor()/method().

- Cara4:
- **new NamaConstructor([parameter]).namaMethod();**
- Cara ini objek diinstansiasi tanpa nama dan langsung digunakan untuk memanggil suatu method() yg non static.

Penggunaan dan Penghancuran Objek

- Objek digunakan untuk memanggil suatu constructor() dan menginisialisasinya.
- Objek juga digunakan untuk memanggil suatu **method([daftarNilaiParameter])** yang non static.
- Teknik yang digunakan java untuk menangani objek yang sudah tidak diperlukan lagi disebut *garbage collection*.
- Objek yang sudah tidak diperlukan lagi akan terdeteksi oleh JVM, kemudian secara otomatis dihancurkan oleh *garbage collector* (bukan oleh programmer).

Keyword this

sebagai Objek standard

- Keyword **this** adalah objek standard milik class itu sendiri, dan **this()** adalah constructor standard class itu sendiri.
- **this** langsung digunakan tanpa didahului proses deklarasi dan instansiasi.
- Salah satu penggunaan objek standard ini bila ada ***attribute non static*** dari suatu class akan **diakses** oleh method() yang berada dalam **class yang sama** dengan atrribute tersebut, dan nama attribute tersebut **sama** dengan nama parameter yang dilewatkan dalam method() yang mengakses.
- **this** juga digunakan untuk memanggil method() **non static** pada **class yang sama**.

Inner Class

- Dalam OOP dibolehkan suatu class **disisipkan** ke dalam **class penggunanya** atau ke method **main().**
- Class yang disisipkan ini disebut sebagai **class Inner.**
- Class Inner yang tidak diberi nama disebut **Anonymous class.**
- Class sebagai tempat untuk disisipi disebut **class Outer.**
- Class yang disisipkan ke method main() disebut juga **local class.**

- Jadi ada 3 jenis class Inner, yaitu **nested class**, **local class**, **anonymous class.**

Ketentuan class Inner

1. Class Outer yang mengandung class Inner, bila dikompilasi akan menghasilkan dua file *.class, yaitu **Outer.class** dan **Outer$Inner.class.**
2. Class Inner boleh diberi modifier akses **public**, atau **protected**, atau **default**, ataupun **private.**
3. Untuk mengakses objek **this** pada class Outer digunakan format: **NamaClassOuter.this.**
4. Class Outer ikut bertanggung-jawab dalam instansiasi objek pada class Inner yang **non-static.**

Contoh: misal objek class Outer adalah a. dan objek class Inner adalah b,

- maka sintaks instansiasi yang benar adalah:
- **Outer a = new Outer();**
- **Outer.Inner b = a.new Inner(); //non static**

5. Jika class Inner bersifat **static**, maka objek milik class Inner tsb boleh dibuat sendiri tanpa melibatkan class Outer nya.
   Inner b = new Inner(); //static
6. Class Inner tidak boleh mengakses attribute atau method() **non static** milik class Outer nya.